If you want to use your own encoding for some reason (and honestly, it's hard to think of a good reason), you can use a magic comment at the top of the file to change the encoding for that file. If you override the encoding, you'll do that for all strings in the file:

```
#encoding: utf-8
plain_string = "dog"
puts "Encoding of #{plain_string.inspect} is #{plain_string.encoding}"
utf_string = "δog"
puts "Encoding of #{utf_string.inspect} is #{utf_string.encoding}"
```

produces:

```
Encoding of "dog" is UTF-8
Encoding of "δog" is UTF-8
```

If there's a shebang line, the encoding comment must be after the shebang, but before any actual Ruby code:

```
#! /usr/local/rubybook/bin/ruby
# encoding: utf-16
```

The special constant `__ENCODING__` returns the encoding of the current source file.

Working with Strings

String is probably the largest built-in Ruby class, with more than one hundred standard methods. We won't go through them all here; the online API documentation has a complete list. Instead, we'll look at some common string idioms—things that are likely to pop up during day-to-day programming.

Maybe we've been given a file containing information on a song playlist. For historical reasons (are there any other kind?), the list of songs is stored as lines in the file. Each line holds the name of the file containing the song, the song's duration, the artist, and the title, all in vertical bar–separated fields. A typical file may start like this:

tut_stdtypes/songdata

```
/jazz/j00132.mp3  | 3:45 | Fats     Waller     | Ain't Misbehavin'
/jazz/j00319.mp3  | 2:58 | Louis    Armstrong  | Wonderful World
/bgrass/bg0732.mp3| 4:09 | Strength in Numbers | Texas Red
```

Looking at the data, it's clear that we'll be using some of class String's many methods to extract and clean up the fields before we use them. At a minimum, we'll need to

- break each line into fields
- convert the running times from mm:ss to seconds, and
- remove those extra spaces from the artists' names.

Our first task is to split each line into fields, and String#split will do the job nicely. In this case, we'll pass split a regular expression, /\s*\|\s*/, that splits the line into tokens wherever split finds a vertical bar, optionally surrounded by spaces (more about regular expressions in Chapter 8, Regular Expressions, on page 129). And, because the line read from the file has a trailing newline, we'll use chomp to strip it off each line as we read it.

We'll store details of each song in a Struct that contains an attribute for each of the three fields. (A Struct is simply a data structure that contains a given set of attributes—in this case the title, name, and length, it's a shortcut for declaring a class that only has instance variables but little to no logic.)